Бази даних

Лекція 6

Тематика лекції

- **SQL, частина 3**

  Функції агрегації та NULL

  GROUP BY та HAVING

  Логічний порядок виконання запиту

  Віконні функції та ранжування

Функції агрегації

- **Працюють над множиною рядків і повертають одне значення.**
- **Без GROUP BY: одне значення на всю вибірку. З GROUP BY: одне значення на кожну групу.**
- **COUNT, SUM, AVG, MIN, MAX тощо.**
- **Усі функції агрегації, крім COUNT(*), ігнорують NULL.**

Функції агрегації

```
SELECT AVG(grade) FROM enrolment;
```

Запит повертає середнє по всіх записах на курси, де оцінку вже виставлено: 77.31.
Записи без оцінки (NULL) не враховуються: оцінку мають 194 записи з 251.
Це не середній бал студентів. Середнє від середніх балів студентів 78.11: студент з п’ятьма курсами впливає на AVG(grade) у п’ять разів сильніше, ніж студент з одним курсом.

COUNT і NULL

Вираз	Що рахує
`COUNT(*)`	кількість рядків
`COUNT(column)`	кількість значень, що не є NULL
`COUNT(DISTINCT column)`	кількість унікальних значень, що не є NULL

```
SELECT
    COUNT(*)                        AS enrolments,
    COUNT(grade)                    AS graded,
    COUNT(DISTINCT student_id)      AS students,
    ROUND(AVG(grade), 2)            AS avg_grade
FROM enrolment;
```

enrolments	graded	students	avg_grade
251	194	38	77.31

AVG рахує середнє лише по 194 записах з оцінкою.

Агрегати без значень

```
SELECT COUNT(*), COUNT(grade), SUM(grade), AVG(grade)
FROM enrolment
WHERE course_id = 7;
```

count	count	sum	avg
6	0	NULL	NULL

- На курсі 7 є 6 записів, але жодної оцінки.
- COUNT повертає 0, а SUM, AVG, MIN, MAX повертають NULL.
- COALESCE(SUM(grade), 0) замінює NULL на 0.
- Для AVG така заміна зазвичай некоректна: "оцінок немає" і "середній бал 0" означають різне.

GROUP BY

- Об’єднує рядки з однаковими значеннями вказаних колонок в одну групу.
- Функції агрегації обчислюються в кожній групі окремо.
- Результат містить один рядок на кожну групу.
- Рядки з NULL у колонці групування утворюють окрему групу.

GROUP BY

```
SELECT group_column, aggregation(...)
FROM table
GROUP BY group_column;
```

GROUP BY

```
SELECT course_id, AVG(grade)
FROM enrolment
GROUP BY course_id;
```

Запит повертає середній бал по кожному курсу.

student_id	course_id	grade
1	1	80
2	1	90
3	2	95
3	1	85

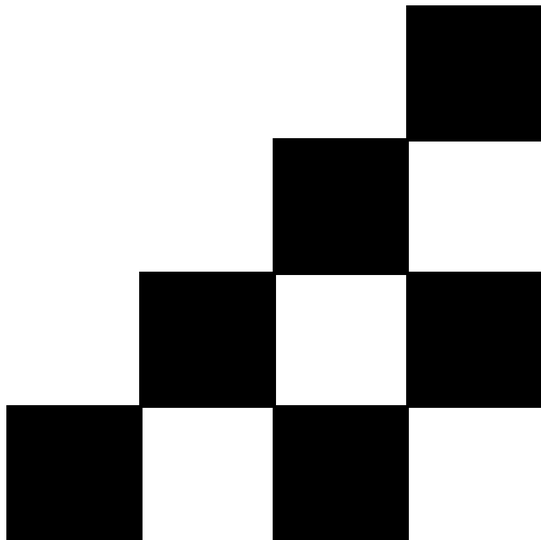
->

course_id	avg
1	85
2	95

```
SELECT course_id, AVG(grade)
FROM enrolment
GROUP BY course_id;
```

course_id = 1: 80, 90, 85 -> 85

course_id = 2: 95 -> 95



GROUP BY: кілька колонок

```
SELECT course_id, enrolment_status, COUNT(*) AS enrolments
FROM enrolment
WHERE course_id IN (5, 8)
GROUP BY course_id, enrolment_status
ORDER BY course_id, enrolment_status;
```

course_id	enrolment_status	enrolments
5	started	7
5	finished	21
8	not_started	7
8	started	15
8	finished	9

Група визначається комбінацією значень усіх колонок з GROUP BY.

GROUP BY разом з JOIN

```
SELECT c.course_id, c.course_name,
       COUNT(*)                AS enrolments,
       COUNT(e.grade)          AS graded,
       ROUND(AVG(e.grade), 2) AS avg_grade
FROM enrolment e
    INNER JOIN course c ON c.course_id = e.course_id
GROUP BY c.course_id
ORDER BY avg_grade DESC NULLS LAST;
```

Спочатку таблиці з'єднуються, потім результат групується.

c.course_name можна вибрати, бо в GROUP BY є первинний ключ c.course_id.

course_id	course_name	enrolments	graded	avg_grade
12	Хмарні технології	6	6	86.00
6	Операційні системи	13	13	81.38
8	Машинне навчання	31	9	80.78
...	...	...	...	...

Що можна вказувати в SELECT

- **Колонку з GROUP BY або вираз над такими колонками.**
- **Функцію агрегації.**
- **Будь-яку колонку таблиці, первинний ключ якої є в GROUP BY (функціональна залежність).**
- **Те саме правило діє для HAVING і ORDER BY.**

Що можна вказувати в SELECT

```
SELECT course_id, grade, AVG(grade)
FROM enrolment
GROUP BY course_id;
```

```
ERROR:  column "enrolment.grade" must appear in the GROUP BY
        clause or be used in an aggregate function
```

- **У групі курсу багато оцінок, і СУБД не знає, яку з них показати.**
- **Показати окрему оцінку поруч із середнім по групі дозволяють віконні функції.**

JOIN розмножує рядки

```
SELECT c.course_name,
       COUNT(e.student_id)    AS students,
       COUNT(ct.professor_id) AS professors
FROM course c
    JOIN enrolment e        ON e.course_id = c.course_id
    JOIN course_teacher ct ON ct.course_id = c.course_id
WHERE c.course_id IN (1, 2)
GROUP BY c.course_id;
```

course_name	students	professors
Програмування	70	70
Математичний аналіз	35	35

35 студентів × 2 викладачі = 70 пар

3 COUNT(DISTINCT ...):

course_name	students	professors
Програмування	35	2
Математичний аналіз	35	1

- **Для SUM і AVG DISTINCT не підходить: два різні значення можуть збігтися.**
- **Надійніше агрегувати кожну таблицю окремо (підзапити, лекція 7) або з’єднувати лише потрібні таблиці.**

HAVING

```
SELECT course_id, AVG(grade) AS avg_grade
FROM enrolment
WHERE avg_grade > 80
GROUP BY course_id;
```

```
ERROR:  column "avg_grade" does not exist
```

Варіант WHERE AVG(grade) > 80 теж не працює:

```
ERROR:  aggregate functions are not allowed in WHERE
```

WHERE виконується до групування і до SELECT: ні груп, ні псевдоніма avg_grade на цьому кроці ще немає.

HAVING

```
SELECT course_id, ROUND(AVG(grade), 2) AS avg_grade
FROM enrolment
GROUP BY course_id
HAVING AVG(grade) > 80;
```

course_id	avg_grade
6	81.38
8	80.78
12	86.00

У HAVING повторюють вираз AVG(grade): псевдонім із SELECT тут не видно. В ORDER BY псевдонім працює.

HAVING

- **WHERE фільтрує рядки ДО групування.**
- **HAVING фільтрує групи ПІСЛЯ групування та агрегації.**
- **Умову на окремі рядки варто розміщувати у WHERE: рядки відкидаються раніше, і запит легше читати.**

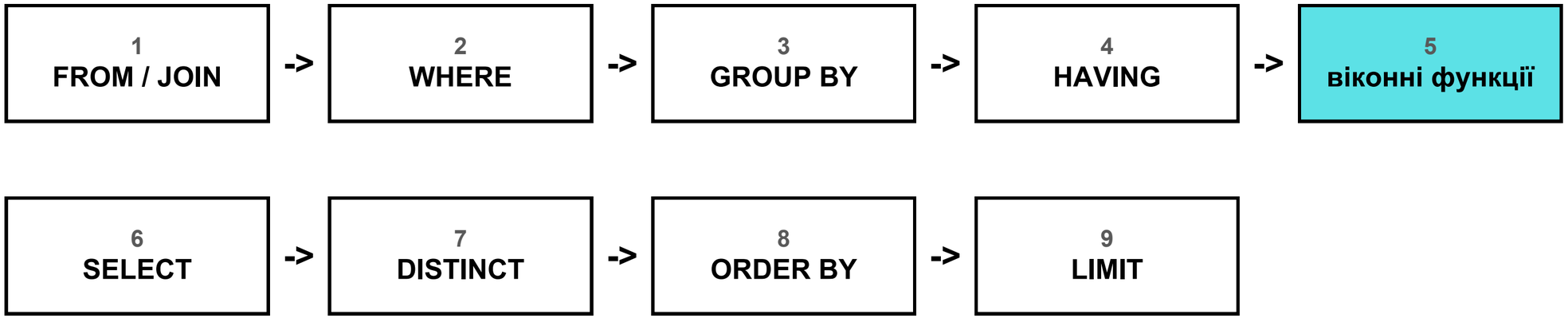
Умовна агрегація: FILTER

```
SELECT course_id,
       COUNT(*)                                  AS enrolments,
       COUNT(*) FILTER (WHERE grade >= 90) AS excellent,
       COUNT(*) FILTER (WHERE grade < 60)  AS failed
FROM enrolment
WHERE course_id IN (1, 2, 3)
GROUP BY course_id;
```

course_id	enrolments	excellent	failed
1	35	11	2
2	35	11	5
3	35	9	4

Кілька агрегатів з різними умовами в одному запиті.

Логічний порядок виконання

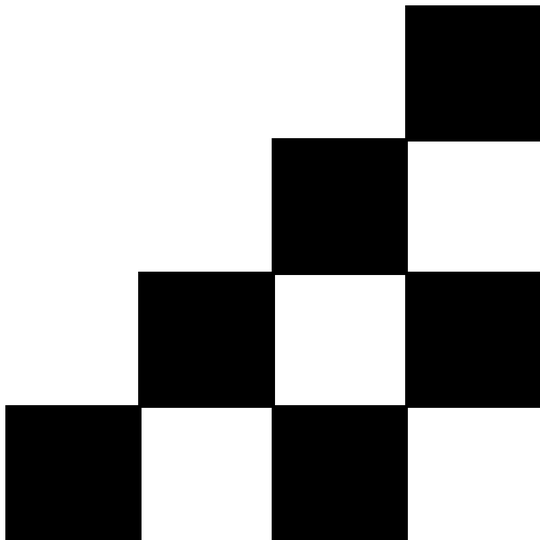


- Агрегати не можна використовувати у WHERE: групи ще не сформовано.
- Псевдонім із SELECT не видно у WHERE і HAVING, але видно в ORDER BY.
- Віконні функції бачать уже згруповані рядки, але їх не можна використовувати у WHERE чи HAVING.

Усе разом

```
SELECT c.course_name, ROUND(AVG(e.grade), 2) AS avg_grade
FROM enrolment e
    INNER JOIN course c ON c.course_id = e.course_id
WHERE c.course_status = 'active'
GROUP BY c.course_id
HAVING AVG(e.grade) > 80
ORDER BY avg_grade DESC;
```

course_name	avg_grade
Операційні системи	81.38
Машинне навчання	80.78



Віконні функції

- **Використовуються, коли потрібні одночасно окремі рядки та результати агрегації.**
- **GROUP BY згортає кожну групу в один рядок.**
- **Віконна функція залишає всі рядки й додає до кожного значення, обчислене по його вікну.**

Віконні функції

- **FUNCTION(...) OVER (...): OVER визначає вікно, тобто набір рядків для обчислення.**
- **OVER (): усі рядки результату після WHERE, GROUP BY, HAVING.**
- **OVER (PARTITION BY column): окреме вікно для кожної групи, усі рядки зберігаються.**
- **OVER (ORDER BY column): порядок рядків у вікні. Потрібен для ранжування, а агрегати з ним стають накопичувальними.**

Віконні функції: приклад

Потрібно показати оцінку кожного студента поруч із середнім балом по предмету.

Віконні функції: OVER ()

```
SELECT p.first_name || ' ' || p.last_name AS full_name,
       c.course_name, e.grade,
       AVG(e.grade) OVER () AS avg_grade
FROM enrolment e
    JOIN student s ON s.student_id = e.student_id
    JOIN person p  ON p.person_id = s.person_id
    JOIN course c  ON c.course_id = e.course_id
ORDER BY full_name, c.course_name;
```

full_name	course_name	grade	avg_grade
Ігор Пилипенко	Дискретна математика	97	77.31
Ігор Пилипенко	Математичний аналіз	73	77.31
Ігор Пилипенко	Програмування	53	77.31
...	...	...	...

У кожному рядку одне й те саме число: середнє по всіх оцінках усіх курсів.

Віконні функції: PARTITION BY

```
SELECT p.first_name || ' ' || p.last_name AS full_name,
       c.course_name, e.grade,
       AVG(e.grade) OVER (PARTITION BY e.course_id) AS avg_course_grade
FROM enrolment e
    JOIN student s ON s.student_id = e.student_id
    JOIN person p  ON p.person_id = s.person_id
    JOIN course c  ON c.course_id = e.course_id
WHERE e.course_id IN (11, 12)
ORDER BY c.course_name, e.grade DESC;
```

full_name	course_name	grade	avg_course_grade
Денис Онищенко	Кібербезпека	92	79.83
...	...	...	...
Юлія Ярошенко	Кібербезпека	49	79.83
Владислав Карпенко	Хмарні технології	100	86.00
...	...	...	...

Середній бал курсу однаковий для всіх рядків курсу, і кожен рядок зберігається.

Вікно і WHERE

```
SELECT e.student_id, e.course_id, e.grade,
       ROUND(AVG(e.grade) OVER (PARTITION BY e.course_id), 2) AS avg_course_grade
FROM enrolment e
WHERE e.student_id = 1
ORDER BY e.course_id;
```

student_id	course_id	grade	avg_course_grade
1	1	61	61.00
1	2	94	94.00
1	3	77	77.00
...	...	...	...

Вікно і WHERE

```
SELECT e.student_id, e.course_id, e.grade,
       ROUND(AVG(e.grade) OVER (PARTITION BY e.course_id), 2) AS avg_course_grade
FROM enrolment e
WHERE e.student_id = 1
ORDER BY e.course_id;
```

student_id	course_id	grade	avg_course_grade
1	1	61	61.00
1	2	94	94.00
1	3	77	77.00
...	...	...	...

WHERE виконується раніше за вікно: у партиції кожного курсу залишився один рядок, і "середнє по курсу" дорівнює оцінці самого студента.

ORDER BY у вікні

```
SELECT e.student_id, e.grade,
       ROUND(AVG(e.grade) OVER (), 2)                  AS avg_all,
       ROUND(AVG(e.grade) OVER (ORDER BY e.grade), 2) AS avg_running
FROM enrolment e
WHERE e.course_id = 12
ORDER BY e.grade;
```

student_id	grade	avg_all	avg_running
35	71	86.00	71.00
34	77	86.00	74.00
32	86	86.00	78.00
33	91	86.00	83.20
36	91	86.00	83.20
31	100	86.00	86.00

- **З ORDER BY агрегат рахується по рамці: від першого рядка партиції до поточного.**
- **Рядки з однаковим значенням сортування потрапляють у рамку разом (дві оцінки 91).**
- **ORDER BY у вікні не сортує результат запиту.**

Функції ранжування

- **ROW_NUMBER(): послідовний номер рядка в партиції згідно з ORDER BY вікна, без повторів.**
- **RANK(): однакові значення мають однаковий ранг, після них у нумерації є пропуск.**
- **DENSE_RANK(): однакові значення мають однаковий ранг, без пропусків.**
- **Без ORDER BY у вікні ранжування не має сенсу: RANK() OVER () повертає 1 для всіх рядків.**

Ранжування на однакових значеннях

```
SELECT name, grade,
       ROW_NUMBER() OVER w AS row_number,
       RANK()       OVER w AS rank,
       DENSE_RANK() OVER w AS dense_rank
FROM (VALUES ('A', 95), ('B', 90), ('C', 90), ('D', 85)) AS t(name, grade)
WINDOW w AS (ORDER BY grade DESC)
ORDER BY grade DESC, name;
```

name	grade	row_number	rank	dense_rank
A	95	1	1	1
B	90	2	2	2
C	90	3	2	2
D	85	4	4	3

RANK пропускає 3, DENSE_RANK не пропускає.

Ранжування і NULL

- При ORDER BY grade DESC записи без оцінки опиняються на початку й отримують ранг 1.
- На курсі 8 оцінку мають 9 записів з 31: 22 студенти без оцінки отримали б ранг 1.

```
RANK() OVER (ORDER BY grade DESC)              -- NULL першими
RANK() OVER (ORDER BY grade DESC NULLS LAST)   -- NULL останніми
```

- ROW_NUMBER() при однакових значеннях нумерує рядки в довільному порядку. Для стабільного результату варто додавати унікальну колонку: ORDER BY grade DESC, student_id.

GROUP BY і віконна функція разом

```
SELECT sg.group_name, p.first_name || ' ' || p.last_name AS full_name,
       ROUND(AVG(e.grade), 2) AS student_avg_grade,
       RANK() OVER (PARTITION BY sg.group_id ORDER BY AVG(e.grade) DESC NULLS LAST) AS student_rank
FROM student s
    JOIN student_group_assignment sga ON sga.student_id = s.student_id
                                     AND sga.assignment_status = 'active'
    JOIN student_group sg ON sg.group_id = sga.group_id
    JOIN person p         ON p.person_id = s.person_id
    LEFT JOIN enrolment e ON e.student_id = s.student_id
GROUP BY sg.group_id, sg.group_name, s.student_id, p.first_name, p.last_name
ORDER BY sg.group_name, student_rank;
```

group_name	full_name	student_avg_grade	student_rank
ІМ-21	Інна Корненко	87.00	1
ІМ-21	Леся Павленко	84.83	2
...	...	...	...

Спочатку GROUP BY рахує AVG для кожного студента, потім вікно ранжує студентів у межах групи.

Фільтрація за результатом вікна

```
SELECT e.course_id, e.student_id, e.grade
FROM enrolment e
WHERE RANK() OVER (PARTITION BY e.course_id ORDER BY e.grade DESC) = 1;

ERROR:  window functions are not allowed in WHERE
```

- **Віконні функції обчислюються після WHERE і HAVING.**
- **Рішення: обчислити вікно у підзапиті, а відфільтрувати в зовнішньому запиті.**

Фільтрація через підзапит

```
SELECT ranked.course_name, ranked.full_name, ranked.grade
FROM (
    SELECT c.course_name, p.first_name || ' ' || p.last_name AS full_name, e.grade,
           RANK() OVER (PARTITION BY e.course_id ORDER BY e.grade DESC NULLS LAST) AS rnk
    FROM enrolment e
        JOIN student s ON s.student_id = e.student_id
        JOIN person p  ON p.person_id = s.person_id
        JOIN course c  ON c.course_id = e.course_id
    WHERE e.grade IS NOT NULL
) AS ranked
WHERE ranked.rnk = 1
ORDER BY ranked.course_name, ranked.full_name;
```

course_name	full_name	grade
Дискретна математика	Леся Павленко	99
Дискретна математика	Поліна Захарченко	99
Дискретна математика	Тарас Шевчук	99

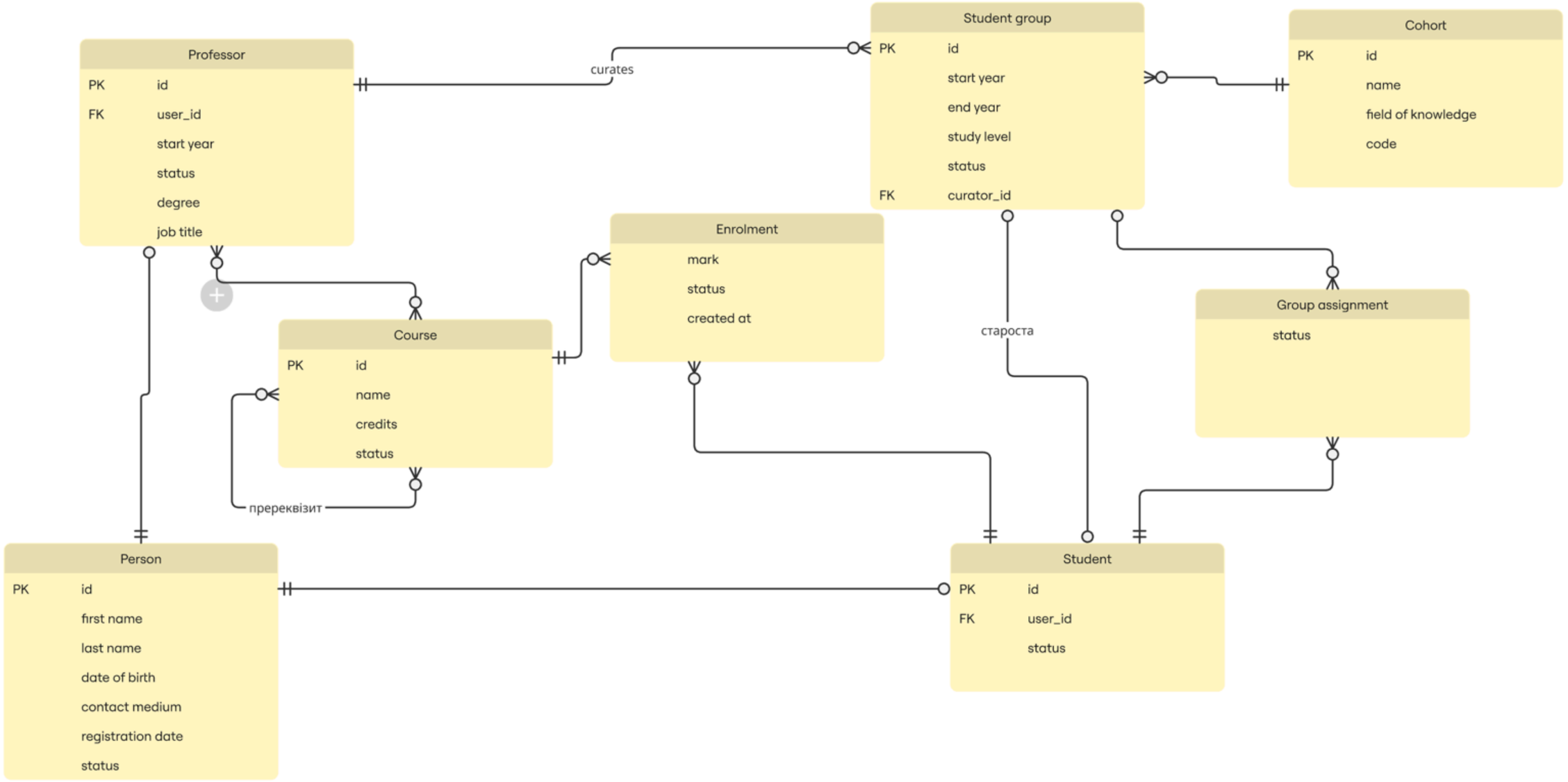
RANK залишає всіх студентів з найвищим балом: на "Дискретній математиці" їх троє.

GROUP BY і віконні функції

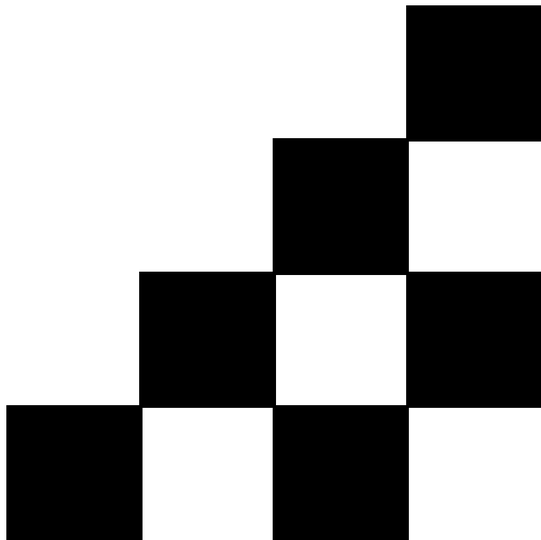
Ознака	GROUP BY	Віконна функція (OVER)
Результат	один рядок на групу	усі рядки, до кожного додано значення
Доступ до окремих рядків	немає, група згорнута	є, з порядком через ORDER BY у вікні
Фільтрація за результатом	HAVING	лише через підзапит або CTE
Кілька групувань в одному запиті	ні	так, кожна функція має власне вікно

Їх можна поєднувати: віконна функція обчислюється над результатом GROUP BY.

Campus


Professor
PK id
FK user_id
start year
status
degree
job title
curates
Student group
PK id
start year
end year
study level
status
FK curator_id
Cohort
PK id
name
field of knowledge
code
Enrolment
mark
status
created at
Course
PK id
name
credits
status
пререквізит
старостa
Group assignment
status
Person
PK id
first name
last name
date of birth
contact medium
registration date
status
Student
PK id
FK user_id
status

Практика
SQL запитів



Запитання